# Cascaded YOLO-based approach for Bangladeshi license plate identification and number recognition

## The paper, our re-run on dataset v1.1, how it fits our research, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 22 October 2026 · replaces the first report of 19 Oct (kept as `Paper02_Study_Report_v1.0_run.pdf`)
Paper: P. Sarker and S. M. Shamsul Alam (Khulna University), *PLOS One* 21(8): e0355858, 10 Aug 2026, DOI 10.1371/journal.pone.0355858, 42 pages, open access (CC BY 4.0). Page numbers are journal pages.
Code and data: github.com/prosensarker4/BLPD (weights, data, a validation notebook; no training script).
Our run: `paper02_kaggle.ipynb` on Tesla T4 x2, 21 Oct 2026, dataset **v1.1** (confirmed in `environment.json`), 3.2 h; full output in `After train/results.zip`.

### The whole report on one page

**The paper in three lines.** Three YOLOv12 models run one after another: a COCO model finds the vehicle, a plate model finds the plate inside it, and a 104-class model finds every digit, letter, "Metro" and whole district name inside the plate. Ordering those labels gives the plate text with no OCR engine. On 150 external photos it reads 95.33 % of plates exactly, against 15.33 % for an OCR pipeline (p.34, p.38).

**Is the v1.1 training OK? Yes.** All 5 jobs ended with `code=0`, every epoch ran (60/60 and 70/70), no NaN, 0 corrupt images, dataset version 1.1 confirmed. Two checks show the pipeline is repeatable: the plate detector (whose data did not change in v1.1) gives the same test scores as on 19 Oct, and the authors' weights give the same result to the last photo (160 of 204).

**What v1.1 changed.** v1.1 cut 91 BLPD "plate crops" that were really whole vehicle photos down to the plate. Retrained on the fixed crops, our character reader reads **87.7 %** of the 651 test plates exactly (v1.0: 83.6 %). On the *unchanged* 204 end-to-end photos the cascade improves from 83.3 to **85.3 %** (and from 82.8 to **86.8 %** without the vehicle stage).

**Four findings.** (1) **The paper reproduces:** its own weights read 95.14 % of its own photos in our run (paper 95.33 %). (2) **Same photos, two models:** in daylight ours and theirs are statistically tied (93.1 vs 95.1 %, McNemar p = 0.58); at night ours reads 66.7 % vs their 38.3 % ($p < 0.001$); over all 204 photos ours is better (85.3 vs 78.4 %, p = 0.02). (3) **The vehicle stage is not needed:** without it accuracy does not drop (86.8 vs 85.3 %, p = 0.55) and the pipeline is 2.6× faster (36 vs 93 ms per photo on a T4). (4) **Both open questions of the first report are solved:** "7.1 FPS" is an averaging artefact (the detector runs at 131.6 FPS with TensorRT), and the low box score mAP50-95 = 46.2 comes from one source, ridoykj, whose boxes are drawn tighter (64.3 without it; paper: 68.5).

**What to tell the faculty in one sentence.** "We reproduced the best published Bangla system on its own photos, matched it in daylight, beat it clearly at night, made it 2.6 times faster by removing a stage, and showed that fixing 91 bad dataset images alone added 2-4 points."

### How this report is organised

| Part | Sections | Read it when |
|---|---|---|
| A. Understand the paper | 1-7: problem, terms, data, method, results, critique, what breaks on our plates | before you explain the paper |
| B. Our v1.1 experiment | 8-14: what we ran, is it OK, results vs paper, v1.0 vs v1.1, six findings, targets | before you quote any number |
| C. Use it | 15-21: research use, benefits, faculty slides, script, Q&A, next steps, GitHub | before the faculty meeting |

# 1. What problem does the paper solve?

Bangladesh had 5,759,353 registered motor vehicles by 30 June 2023, 72 % of them motorcycles (BRTA, p.2). Traffic police, toll gates and parking need to read plates automatically. A Bangladeshi plate has four required parts: a **city (district) code** written as a Bangla word, a **vehicle-type letter**, a **two-digit class number** and a **four-digit registration number**. Plates of metropolitan areas add the word **"Metro"** after the city (p.2, Fig. 2).

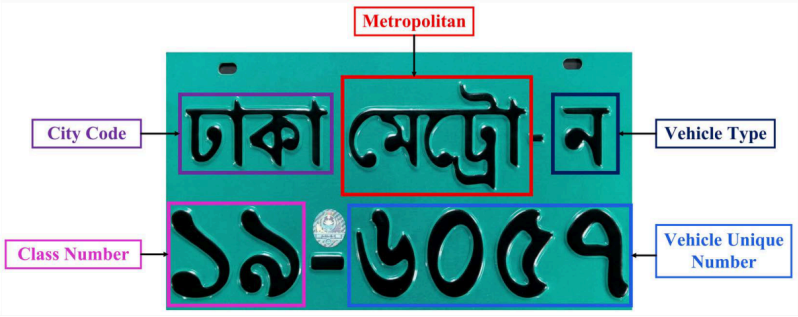



Figure 1. Parts of a standard Bangladeshi plate: city code, "Metro", vehicle type, class number and unique number (paper Fig. 2, p.3). Figure from Sarker & Alam (2026), PLOS One, CC BY 4.0.

Earlier Bangla systems had two problems (p.4-5, Table 1):

- **Too few classes.** Character recognizers used 14 to 60 classes. Real plates need more than 100 (10 digits, about 30 letters, "Metro" and 64 district names).
- **General OCR engines fail on Bangla.** Tesseract and EasyOCR misread conjunct consonants (*jukto borno*) and the two-line layout. Reported OCR accuracy was 78 % (EasyOCR) and 22-73 % (Tesseract).

The paper's answer: treat every piece of the plate, even a whole district name, as an **object** that a YOLO detector can find. Then no OCR engine is needed (p.5).

# 2. Key terms in plain words

| Term | Meaning |
|---|---|
| Cascade | models run one after another; each one only looks inside the crop given by the one before |
| LPD / LPNR | License Plate Detection model (finds the plate) / License Plate Number Recognition model (finds the characters) |
| OCR-free | the text comes from a detector's class labels, not from an OCR engine such as Tesseract or EasyOCR |
| Class / token | one thing the detector can name: a digit, a letter, "Metro" or a district word such as "Dhaka" |
| Precision / Recall | of all boxes the model drew, how many were right / of all real objects, how many it found |
| mAP50 | mean average precision when a box counts as correct if it overlaps the true box by at least 50 % (IoU 0.5) |
| mAP50-95 | the same, averaged over stricter overlaps 0.50, 0.55 ... 0.95; it rewards boxes drawn *exactly like the labels* |
| Full-number exact match | the whole plate text is right; one wrong character = wrong plate |
| Character-wise accuracy | share of characters that are right |
| Levenshtein similarity | 1 - (edits needed to turn the prediction into the truth / length of the truth); a nearly right plate scores near 100 % |
| McNemar test | compares two models on the same photos using only the photos where they disagree; $p < 0.05$ = a real difference, not luck |
| External validation | a test on photos that were never used for training or tuning |
| Latency / FPS | time to process one photo (ms) / photos per second = 1000 / latency |
| Near-duplicate | a different file that shows the same photo or plate (Roboflow copy, re-post, next video frame); see paper #23 |

# 3. The data

| Set | Images | Train / val / test | Classes | Boxes | Page |
|---|---|---|---|---|---|
| Dataset-01 (plates, for LPD) | 2,140 | 1,502 / 425 / 213 | 1 | 2,251 (Fig. 3 shows 1,561) | p.6-7, Table 2 |
| Dataset-02 (plate crops, for LPNR) | 7,217 | 5,058 / 1,438 / 721 | 104 | 63,879 | p.6-10, Tables 2-3 |
| External validation | 150 | test only | - | - | p.32 |

- **Sources:** their own phone photos in Khulna (Gollamari, Sonadanga, Zero Point), all in **daylight and sunny weather**, plus web crawling and Roboflow (p.6).
- **Split:** 70 / 20 / 10, done by image. No de-duplication step is described.
- **Strong imbalance:** digit "1" has 7,396 boxes, "Metro" 6,292, "Dhaka" 4,947, but Munshiganj has 2, Nowabganj 11, Bagerhat 37, Joypurhat and Kustia 45 (p.9).
- **Class list:** some "different" classes are the same token written two ways: "Cox-s Bazar" (47) and "Cox-s-Bazar" (58), "Da" (402) and "DA" (141), "Tha" (405) and "THA" (179) (Table 3, p.10). So 104 classes are really about 101 tokens. Our cleaned vocabulary has 101.

# 4. The method

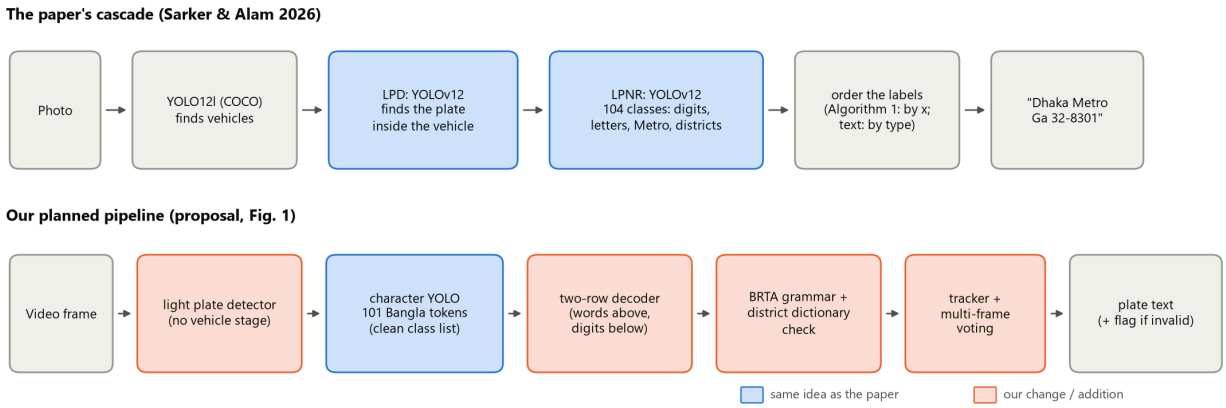



Figure 2 (drawn for this report). Top: the paper's cascade (Algorithm 1, p.15-16). Bottom: our planned pipeline from the proposal. Blue = idea we keep, orange = what we change or add.

| Stage | What it does | Model and settings |
|---|---|---|
| 1. Vehicle | finds cars, buses, trucks and motorcycles; each vehicle box is cropped | Ultralytics YOLO12l pretrained on COCO, not retrained (26.4 M parameters, 88.9 GFLOPs) (p.15) |
| 2. Plate (LPD) | finds the plate inside the vehicle crop; the box is mapped back to the photo | YOLOv12 trained on Dataset-01, 100 epochs (p.15) |
| 3. Characters (LPNR) | finds digits, letters, "Metro" and district words in the plate crop | YOLOv12 trained on Dataset-02, 70 epochs (p.16) |
| 4. Decoding | Algorithm 1: "sort C by left-to-right order" and join (p.16). The text below it: labels are "organized as City-Metro (if available)-Character-Number", e.g. "Dhaka Metro Ga 328301" (p.16) | simple rule, no language model |

**Training settings (Table 5, p.18):** 640×640 input, batch 16, optimizer "auto", lr0 0.01, weight decay 0.0005, NMS IoU 0.7, data augmentation "No", no cosine schedule, deterministic, no early stopping. Training ran on a Colab Tesla T4: LPD 2.42 h (87.3 s per epoch), LPNR 4.35 h (223.6 s per epoch) (p.17-18, p.24). The model size (n, s, m...) is not stated; the repo's files suggest a medium LPD (40.8 MB) and a nano LPNR (5.7 MB).

**Why YOLOv12?** It adds attention ("area attention" and R-ELAN blocks) to YOLO and was the most accurate YOLO for its speed when the work was done (p.13). On their character data it beat YOLOv8, v9, v10 and v11 (Table 14, p.37).

# 5. Every result in the paper

| Result | Value | Page |
|---|---|---|
| LPD validation (425 images) | P 0.993, R 0.970, mAP50 0.974, mAP50-95 0.674 | p.21, Table 6 |
| **LPD test** (213 images, 226 plates) | **P 0.986, R 0.965, mAP50 0.963, mAP50-95 0.685**; 219 found, 7 missed, 6 false | p.22-23, Table 7 |
| LPNR validation (1,438 images) | P 0.952, R 0.905, mAP50 0.950, mAP50-95 0.770 | p.24, Table 8 |
| **LPNR test** (721 images, 6,292 boxes) | **P 0.950, R 0.898, mAP50 0.949, mAP50-95 0.767**; digits mAP50 ≥ 0.985; weakest "Sa" 0.346, Joypurhat 0.42, "Pa" 0.60 (mAP50-95) | p.25-31, Table 9 |
| YOLO versions on Dataset-02 (test mAP50) | v8 0.936, v9 0.938, v10 0.897, v11 0.931, **v12 0.949** | p.37, Table 14 |
| **External 150 photos** | **plate found 100 %, full number exact 95.33 % (143/150), characters 98.82 %, Levenshtein 99.57 %** | p.34, Table 10 |
| OCR pipeline on the same 150 | 23 correct = 15.33 % (counted "ignoring minor mismatches") | p.37-38, Table 15 |
| Laptop CPU (i5-8265U), per photo | mean 2,311.53 ms (vehicle 911 + plate 1,091 + characters 233), max 6,438 ms | p.35, Table 11 |
| Desktop GPU (RTX 3050), per photo | mean 117.16 ms (39.6 + 45.2 + 19.5) ≈ 8.5 FPS, max 284.31 ms | p.35, Table 12 |
| Jetson Orin Nano, CPU only, per photo | mean 10.37 s, max 25.44 s | p.36, Table 13 |




Figure 3. The 7 external photos the cascade read wrongly; the paper calls every error partial, and the plate was found in all of them (paper Fig. 30, p.34). Figure from Sarker & Alam (2026), PLOS One, CC BY 4.0.

# 6. Critical review

### Strengths

- **Peer reviewed and recent** (PLOS One, Aug 2026), with data, weights and a validation notebook public. That is rare for Bangla LPR.
- **Good idea:** word-level classes for district names avoid the hardest OCR problem (conjunct letters), and 104 classes cover nearly every plate.
- **Several end-to-end metrics** (exact, character-wise, Levenshtein) and **per-stage latency on three devices**, with mean, std, min and max.
- **Honest about limits:** the conclusion names low light, weather, blur, occlusion, imbalance and lightweight models as future work (p.41).
- **Its numbers reproduce on its own data:** Table 10 in our Kaggle run, Tables 6, 7 and 9 in a local retraining (section 10).

### Weaknesses (with page numbers, so you can show them)

| # | Problem | Why it matters |
|---|---|---|
| 1 | **Test sets leak.** Split by image from Roboflow/web sources with no de-duplication. Our audit: 94 of the 213 LPD test photos (44.1 %) and 270 of the 721 LPNR test crops (37.4 %) have a near-duplicate in their own train set; 345 test crops show a plate whose text is also in train. | Tables 7 and 9 are partly measured on seen images, so they are optimistic for new photos (paper #23). |
| 2 | **The datasets are smaller than they look.** In our cleaning, only 185 of the 7,217 character crops survived: 4,894 were duplicates, 2,019 had masked digits, mosaics or incomplete labels, 119 were synthetic templates. Of 2,140 plate photos, 1,001 were duplicates (Figure 4). | "9,357 images" overstates how much distinct data the models saw. |
| 3 | **Max and mean are mixed.** The abstract and Table 17 give 284.31 ms (GPU) and 25.439 s (Jetson), which are the *maximum* times, next to the *mean* CPU time 2,311.53 ms (p.1, p.39). | The GPU mean is 117.16 ms; readers can mis-compare. |
| 4 | **"Edge deployment on Jetson"** is claimed as a contribution (p.6), but the Jetson ran on its CPU only, at 10.4 s per photo (p.17, p.36). | Not real time; the Jetson GPU was never used. |
| 5 | **Daylight only.** Own photos were all taken in daylight and sunny weather (p.6). | Our run: 38 % exact at night with their weights. |
| 6 | **Decoding is described two ways.** Algorithm 1 sorts all boxes left to right (p.16); done literally on two-line plates this reads **1 of 651** of our test plates. The text orders labels by type (city, Metro, letter, number), which works. | Coding Algorithm 1 as written gives almost 0 %. |
| 7 | **Small, self-chosen external set** (150 photos, "online and offline sources", p.32); 6 of the 150 are near-duplicates of other photos in our check. | One external set cannot show generalisation. |
| 8 | **OCR baseline not described:** the OCR engine and its settings are not named (p.37). | The 15.33 % cannot be reproduced. |
| 9 | **Text errors:** section 2.2 describes the original 2016 YOLOv1 (S×S grid, 448 px, fully connected layers), not YOLOv12 (p.10-13); "accuracy" uses true negatives, which do not exist in object detection (p.16); 2,251 vs 1,561 boxes (p.6 vs Fig. 3); duplicate class names (Table 3). | Shows the review was light; do not copy these parts. |
| 10 | **Unequal YOLO comparison:** Table 14 does not give the model size or settings of each version, and each was run once. | The v12 win (0.949 vs 0.936) may be noise or size. |

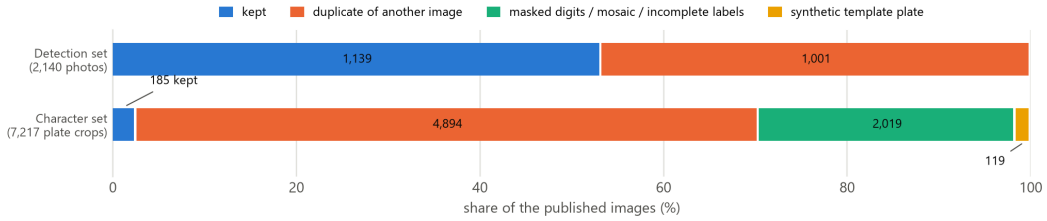



Figure 4. What our de-duplication and label check did to BLPD's published images (dataset/docs/dedupe_report.md). "Duplicate" = a Roboflow copy, a re-post, or the same photo in another public dataset; one copy of each cluster was kept. BLPD's plate boxes are loose, so its kept photos are used only as training hints.

## 7. What would break on our plates (and what we do about it)

| Risk | Why | Our answer |
|---|---|---|
| Rows mixed | a pure left-to-right sort mixes the two lines | two-row decoder: words on top, digits below, each sorted by x |
| Rare districts missed | 2 to 50 training boxes for some districts | merged sources; synthetic rare plates (G6); district dictionary |
| Night, rain, blur, fog | training photos were daylight only | night photos in train; per-condition tests; CLAHE study (RQ4) |
| Vehicle not found | COCO has no CNG/rickshaw class; dark photos | skip the vehicle stage (finding 3) |
| Too slow | three models, 8.5 FPS on an RTX 3050 | two light models, TensorRT/OpenVINO, tracking (RQ3) |
| Wrong but plausible text | no check that the result is a legal plate | BRTA format check; flag or re-read invalid text (O3) |

# 8. What we ran, and what changed in v1.1

The notebook `paper02_kaggle.ipynb` rebuilds the paper's cascade on our **498R BanglaLPR All-in-One** dataset (merged from every public Bangla source, de-duplicated, grouped 70/15/15 split, seed 42, val/test with human labels only) and also runs the **authors' own weights**, so both can be compared on the same photos.

### Dataset v1.1 (20 Oct): the only change that matters here

In v1.0, **91 BLPD rows** of the character set (train 63, val 9, test 19) were not plate crops but whole vehicle photos with tiny character boxes. v1.1 cuts each one to its plate (from the union of its character boxes). Nothing was dropped and the splits did not move. The plate-detection set and the 204 end-to-end photos did not change.

| Job | GPU | What it does | Time |
|---|---|---|---|
| `lpd_v12s` | 0 | YOLO12s plate detector, 60 epochs, 640, batch 16, SGD; train 8,574 photos (4,067 human + 4,507 auto boxes), val 896, test 1,153 (1,011 plates) | 182.2 min |
| `lpnr_v12n` | 1 | YOLO12n character detector, 70 epochs, 640, batch 16, 101 Bangla tokens; train 5,465 crops, val 536, test 467 (boxes); plus 651 test crops with typed plate text | 95.5 min |
| `cascade_ours` | 0 | YOLO12l (COCO) → our LPD → our LPNR on the 204 end-to-end photos, with and without the vehicle stage | 0.8 min |
| `cascade_blpd_weights` | 1 | the same cascade with the authors' LPD_Model.pt and LPNR_Model.pt; also their LPNR on our 651 text crops | 1.2 min |
| `speed_lpd_v12s` | 0 | batch-1 speed of the plate detector: PyTorch FP32/FP16, TensorRT FP16, CPU ONNX | 6.0 min |

**End-to-end photos (204):** 144 of the paper's own 150 external photos (6 near-duplicates removed in our build) and 60 NightLPBD night photos, each with its plate text typed in.

**Differences from the paper:** LPD 60 epochs instead of 100; SGD instead of "auto"; sizes s and n (the paper does not state its sizes); 101 tokens with the duplicate classes merged; our two-row decoder; a different, harder test set. Software: Ultralytics 8.4.170, PyTorch 2.11 + CUDA 12.8, Python 3.13 (Kaggle, 21 Oct 2026 17:03).

# 9. Is the training OK?

| Check | Result | OK? |
|---|---|---|
| Dataset version | `dataset_version: 1.1` in environment.json; folder BanglaLPR_AllInOne_v1_1; counts as expected | yes |
| Jobs | all 5 ended with code=0; total 3.2 h of the 12 h Kaggle limit | yes |
| Epochs | LPD 60 of 60 (best.pt = epoch 36), LPNR 70 of 70 (best.pt = epoch 55); under the 5 h / 4 h caps | yes |
| Data loading | 0 corrupt images in every split; 160 plate-free test photos as expected | yes |
| Curves | no NaN; both validation curves rise and then stay flat (Figure 5); LPD has a short dip at epoch 3 (warm-up) and recovers by epoch 6 | yes |
| Repeatable | LPD test scores identical to the v1.0 run (P 96.7, R 96.2, mAP50 98.9); authors' cascade identical (160 of 204 photos) | yes |
| GPU memory | peak 7.7 GB of 16 GB on the T4 | yes |
| Speed row "7.10 FPS" | an averaging artefact of the PyTorch FP16 loop, not the model (section 13, finding 6) | ignore it |
| LPD mAP50-95 46.2 | caused by one test source with a different box style (finding 4) | explained |

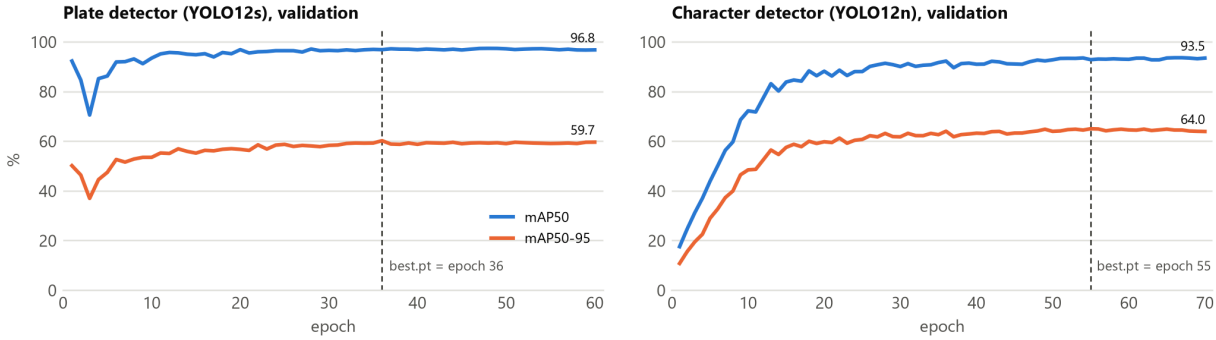



Figure 5. Validation mAP per epoch. Both models converge; the dashed line marks the epoch Ultralytics kept as best.pt (best 0.1·mAP50 + 0.9·mAP50-95 on validation).

## 10. Results next to the paper

| Metric | Paper (page) | Ours v1.0 | Ours v1.1 | Authors' weights (our run) |
|---|---|---|---|---|
| LPD test precision / recall | 98.6 / 96.5 (p.23) | 96.7 / 96.2 | 96.7 / 96.2 | - |
| LPD test mAP50 / mAP50-95 | 96.3 / 68.5 (p.23) | 98.9 / 46.2 | **98.9** / 46.2 (64.3 w/o ridoykj) | - |
| LPNR test boxes mAP50 / mAP50-95 | 94.9 / 76.7 (p.25) | 92.6 / 63.9 | **97.1** / 66.6 | - |
| LPNR on our 651 text crops: plate exact | - | 83.56 | **87.71** | 74.35 |
| Cascade: plate found in photo | 100 (p.34) | 93.14 | 93.14 | 88.24 |
| Cascade: whole plate exact (204 photos) | 95.33 (p.34, 150 photos) | 83.33 | **85.29** | 78.43 |
| Cascade: characters correct | 98.82 (p.34) | 92.59 | 92.78 | 88.18 |
| Cascade: Levenshtein similarity | 99.57 (p.34) | 90.57 | 90.84 | 84.65 |
| Cascade without vehicle stage: exact | - | 82.84 | **86.76** | 77.94 |
| Time per photo, with / without vehicle stage | 117.16 ms, RTX 3050 (p.35) | 85.8 / 31.5 ms | 93.3 / 36.2 ms | 96.4 / 38.9 ms |
| Training time LPD / LPNR | 145 / 261 min (p.17-24) | 183 / 93 min | 178 / 94 min | - |

> **Read this table carefully.** The paper's column is on the paper's data; ours is on the 498R test split, which is larger, de-duplicated and harder. The true head-to-head is "ours" vs "authors' weights" on the *same* 204 photos (finding 2). Times are on one Kaggle T4 (PyTorch, no TensorRT) and vary by about ±10 % between runs.

## 11. What the v1.1 fix changed

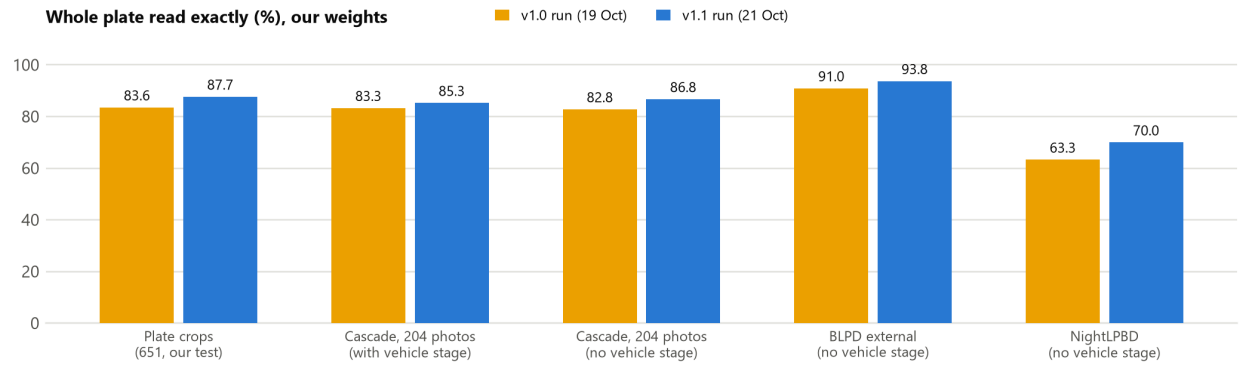



Figure 6. Our weights, v1.0 run vs v1.1 run. Only the character model changed between the runs (the plate detector and the 204 end-to-end photos are the same), so the gain on the four right-hand pairs comes from the fixed crops alone (63 training + 9 validation).

- **On unchanged photos the reader got better:** +4 photos with the vehicle stage (170 → 174 of 204) and +8 without it (169 → 177). Night: 63.3 → 70.0 %. BLPD external: 91.0 → 93.8 % (no vehicle stage).
- **On the 651 text crops** the gain is larger (83.6 → 87.7 %), but here 19 test crops were also fixed, so part of that gain is an easier, now-correct test.
- **Character box mAP50 92.6 → 97.1** is mostly the test fix: in v1.0 the 19 whole-photo test images had tiny character boxes that were almost impossible to hit.
- **We cannot test whether +4 / +8 photos is significant:** the v1.0 per-photo predictions were never downloaded. The direction is the same in every subset.

# 12. Six findings

## Finding 1: the paper reproduces on its own data

| On BLPD's own data | Paper (page) | Our check | Where |
|---|---|---|---|
| Authors' weights, their external photos: whole plate exact | 95.33 (p.34) | **95.14** (137 of 144) | Kaggle, 19 and 21 Oct (same result) |
| Authors' weights, their external photos: characters correct | 98.82 (p.34) | 98.93 | Kaggle |
| Retrained with paper settings on BLPD's split: LPD test mAP50 / mAP50-95 | 96.3 / 68.5 (p.23) | 97.2 / 68.1 | local preview, 13 Oct (RTX 5060 Ti) |
| Retrained: LPNR test mAP50 / mAP50-95 | 94.9 / 76.7 (p.25) | 96.7 / 80.0 | local preview |
| Retrained cascade, 150 external photos: exact / characters | 95.33 / 98.82 (p.34) | 90.67 / 98.05 | local preview |

The printed numbers are real on the paper's own data. This also proves our cascade code and scoring: given the paper's models, they give the paper's number. The official Kaggle version of the retraining check (`paper02_original_check.ipynb`, about 7.5 h) is still to be run.

## Finding 2: same photos, two models: a tie in daylight, a clear win at night

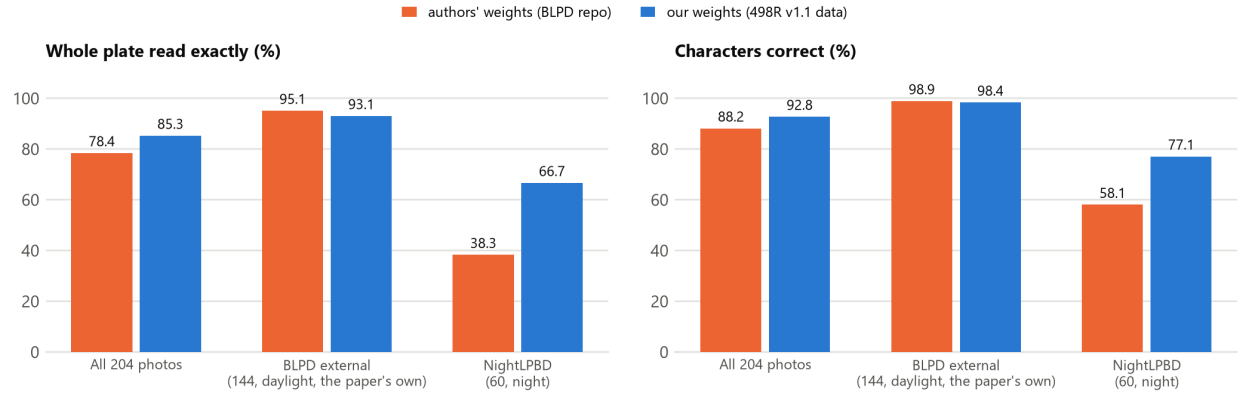



Figure 7. The paper's pipeline (with the vehicle stage), same 204 photos, same code, two sets of weights.

| Same photos (paper's pipeline) | Only ours right | Only authors' right | Both / neither | McNemar p | Verdict |
|---|---|---|---|---|---|
| All 204 photos | 23 | 9 | 151 / 21 | 0.020 | **ours better** |
| BLPD external (144, their photos) | 5 | 8 | 129 / 2 | 0.58 | tie |
| NightLPBD (60) | 18 | 1 | 22 / 19 | 0.00008 | **ours much better** |
| BLPD external, no vehicle stage | 6 | 6 | 129 / 3 | 1.0 | exact tie (135 = 135) |

On the authors' own daylight photos the 2-point gap (95.1 vs 93.1 %) is 3 photos and is not significant; without the vehicle stage both read exactly 135 of 144. At night their model reads 23 of 60 and ours 40 of 60 (characters 58.1 vs 77.1 %). Most of their night failures are photos with no reading at all: 27 of 60 (ours: 14).

## Finding 3: the vehicle stage costs time and adds nothing

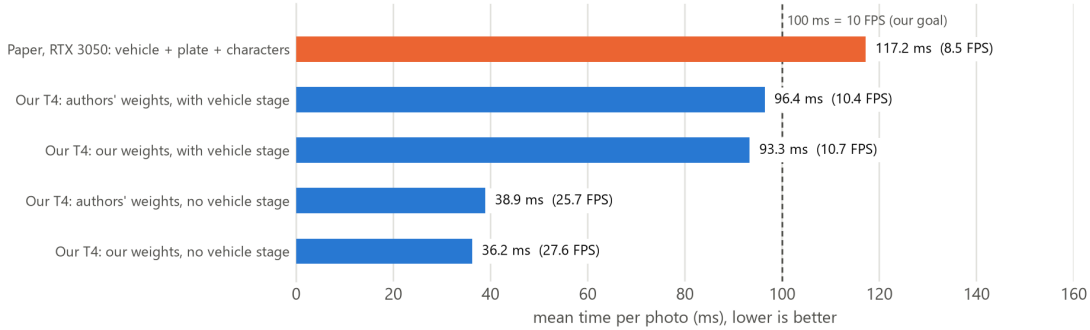



Figure 8. Mean time per photo. The paper's number is on an RTX 3050; ours on one Kaggle T4 (PyTorch, no TensorRT). Removing the COCO vehicle model cuts the time by about 2.6×.

| Our weights, 204 photos | Exact | Characters | Plate found | ms per photo (mean / max) |
|---|---|---|---|---|
| vehicle → plate → characters (paper) | 85.29 % (174) | 92.78 % | 93.14 % | 93.3 / 330.0 |
| plate → characters (no vehicle stage) | **86.76 % (177)** | **94.89 %** | **95.59 %** | **36.2 / 110.8** |

Without the vehicle stage 7 photos become right and 4 become wrong (McNemar p = 0.55, no real difference); 6 of the 7 photos it rescues are night photos, where the COCO model often misses the dark vehicle, so the plate is never searched. The vehicle model is the largest of the three (YOLO12l) and takes about a third of the paper's GPU time (Table 12). Dropping it is the cheapest way to our ≥ 10 FPS goal: about 28 photos per second on a T4 already.

## Finding 4: the low box score (mAP50-95 46.2) is one dataset's box style

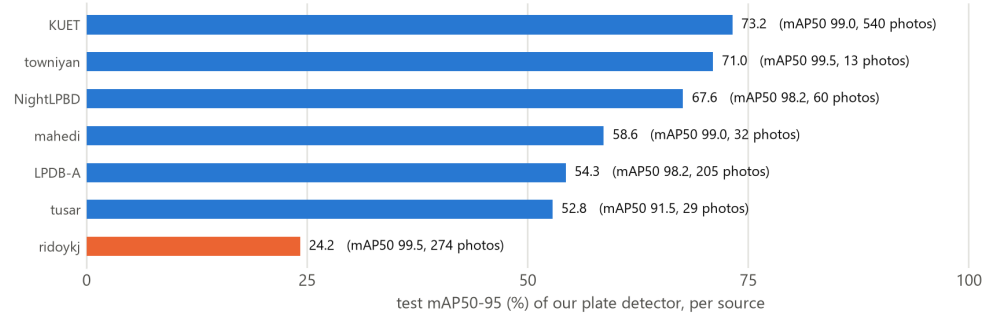



Figure 9. Our plate detector scored per test source (local re-scoring of the Kaggle best.pt; the full test set gives the same 98.9 / 46.2 as on Kaggle).



Figure 10. Green = label box, orange = our detector's box. Top: ridoykj test photos: labels hug the inner plate; our boxes include the plate rim. Bottom: KUET: both agree.

Our detector finds ridoykj plates perfectly (mAP50 99.5), but its boxes are 19 % wider and 36 % taller than ridoykj's labels (median), so at strict overlaps they "miss" (mean IoU 62 %). ridoykj is 274 of the 1,153 test photos (24 %), which pulls the whole test set down to 46.2. Without it, mAP50-95 is **64.3**, close to the paper's 68.5; KUET alone is 73.2. This also explains why validation (60.4) is higher than test: ridoykj is a smaller part of validation. It does not hurt reading: the cascade pads every plate crop by 8 %.

### Finding 5: the decoder and a format check matter as much as the detector

| Our reader on the 651 test crops | Plates | Exact |
|---|---|---|
| Algorithm 1 taken literally: all boxes sorted left to right | 651 | **1 (0.15 %)** |
| Our two-row decoder (words above, digits below), as described in the paper's text | 651 | **571 (87.71 %)** |
| ... readings that pass the BRTA format check (district, [Metro], letter, 6 digits) | 599 | 571 right, 28 wrong |
| ... readings that fail the format check | 52 | **all 52 wrong** |

- **The format check is a free error detector:** it flags 52 of the 80 wrong readings (65 %) and never flags a right one. Flagged plates can be re-read by a second model; in paper #24 this "format fallback" added +1.84 points with no new error (p = 0.0005).
- **What goes wrong:** of the 80 errors, 29 are in the digits, 25 in the letter, 15 in the district word and 11 in several parts. Only 3 wrong readings have mean confidence ≥ 0.9, so confidence is also useful for flagging.
- **By source:** LPDB-A 96.0 %, ANPR 84.8 %, KUET 84.7 %, NightLPBD 80.8 %, BLPD 80.0 % (30 crops). **By condition** (Figure 11): clear 90.4 %, night 83.1 %, blurry 79.5 %, fog 76.9 %, synthetic rain 74.5 %.
- Validation crops (766) score lower, 80.6 %, so do not tune thresholds on test.

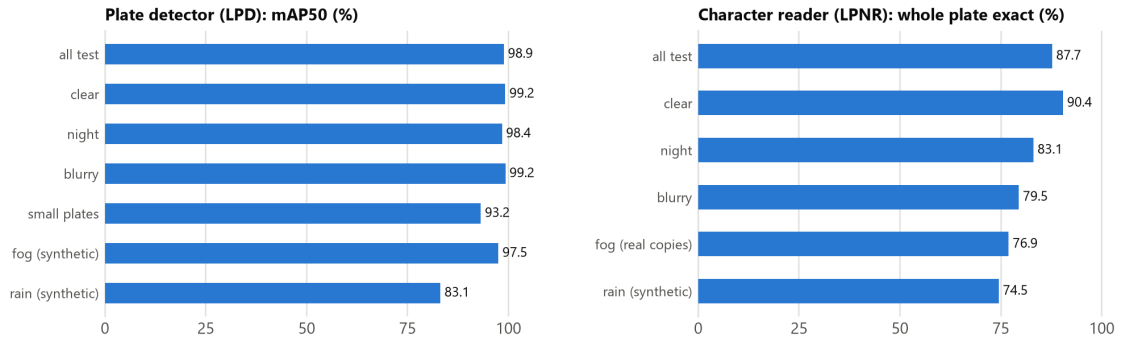



Figure 11. Per-condition test results (fixed team subsets; rain and detection-fog images are synthetic copies).

### Finding 6: real speed numbers, and why "7.10 FPS" is wrong

| Plate detector alone (YOLO12s, 640 px, batch 1) | ms per photo | FPS |
|---|---|---|
| Kaggle T4, PyTorch FP32 | 18.9 | 52.9 |
| Kaggle T4, **TensorRT FP16** | **7.6** | **131.6** |
| Kaggle T4, PyTorch FP16 (the kit's table row) | 140.9 mean, but 17.9 for a single photo | 7.1 (artefact) |
| Kaggle CPU, ONNX FP32 | 166.8 | 6.0 |

The FP16 row averages 200 photos, and a few very slow calls inflate the mean: one photo takes 17.9 ms (1.8 + 15.0 + 1.1 ms), nowhere near 141 ms. Quote TensorRT or FP32. The CPU number matters for our laptop goal: YOLO12s at 640 px gives only 6 FPS on Kaggle's CPU, so the laptop version needs a nano model, a smaller input or OpenVINO. The same FP16 row is used in 7 of the kit's notebooks, so teammates' "FPS" rows can have the same problem.

## 13. Ours vs the proposal's targets

| Proposal target | Our result now (v1.1) | Status |
|---|---|---|
| Detection mAP50 ≥ 95 % | 98.9 % on the 1,153 test photos | met |
| Character accuracy ≥ 90 % | 94.9 % on the 204 end-to-end photos without the vehicle stage (98.5 % daylight, 84.8 % night); 97.9 % on the 651 crops | met (night 84.8 %) |
| ≥ 10 FPS full pipeline on standard hardware | 27.6 photos/s on a T4 without the vehicle stage; detector alone 6 FPS on the Kaggle CPU | GPU yes, CPU not yet |
| Duplicate-free vs standard split (RQ1) | BLPD audit: 44.1 % / 37.4 % of test images duplicated in train | evidence ready |

PART C · USE IT

# 14. How to use this paper in our research

| Our research part | How paper #2 is used | Evidence we now have (v1.1) |
|---|---|---|
| Stage 2: character-level recognizer (O3) | design source: one YOLO detects digits, letters, "Metro" and whole district words; no OCR engine | 87.7 % of plates and 97.9 % of characters right on our 651 test crops |
| Baseline for every table | the authors' public weights, run on our photos with our code | same photos: tie in daylight (p 0.58), 66.7 vs 38.3 % at night (p < 0.001) |
| RQ1: duplicate-free split | BLPD is a second leaky dataset next to KUET (paper #23) | 44.1 % of LPD test and 37.4 % of LPNR test duplicated in train |
| RQ2: ≥ 90 % characters with correction | the literal x-sort is the "no decoder" baseline; the format check is the first correction step | x-sort 0.15 % vs our decoder 87.7 %; format check flags 52 errors, 0 false alarms |
| RQ3 / O4: real time | their 3-model cascade is the slow reference (8.5 FPS RTX 3050, 2.3 s laptop CPU) | no vehicle stage: 2.6× faster, same accuracy; TensorRT detector 131.6 FPS |
| RQ4: robustness | their daylight-only model shows what happens without diverse data | night +28 points with our merged data; per-condition table |
| O1: the benchmark itself | the v1.1 fix is a measured data-quality effect | 91 fixed crops → +4 / +8 photos on unchanged test photos |
| O2: detector choice | their YOLO v8-v12 table is on leaky data | redo it on our split with equal sizes and 3 seeds |
| Thesis writing | Introduction, Related work (their Table 1 lists 14 Bangla systems), Method, Experiments | BibTeX at the end |

# 15. What benefit do you get from it?

- **A strong, citable baseline.** A 2026 peer-reviewed journal paper with public weights. Matching it in daylight and beating it at night, on the same photos and with a significance test, is a clear result examiners understand.
- **Proof that our evaluation code is right and repeatable.** It gives the paper's own number with the paper's weights (95.14 vs 95.33), and the same results in two runs two days apart.
- **A measured data-quality story.** Fixing 91 bad crops raised accuracy on unchanged photos. That supports the whole dataset effort (O1) with a number, not an opinion.
- **Design decisions with evidence:** OCR-free recognition (paper: 95.33 vs 15.33), no vehicle stage (finding 3), a real two-row decoder (finding 5), a format check (finding 5), a clean class list.
- **Honest metrics you can defend:** you can now explain every odd number (mAP50-95 46.2, 7.1 FPS) instead of hiding it.
- **The authors' future work is our project:** low light, weather, blur, occlusion, imbalance and lightweight models (p.41).
- **Reusable assets:** their district/letter class scheme (mapped to our 101 tokens), their metrics, and their per-stage latency reporting.

### A paragraph you can adapt for the related-work chapter

```
Sarker and Alam [ref] cascade three YOLOv12 models (a COCO vehicle detector, a plate
detector and a 104-class character detector that treats district names as single
objects) and report 95.33% full-plate accuracy on 150 external images, against 15.33%
for an OCR pipeline. Their data are daylight-only and split by image; in our audit
44.1% of their detection test images and 37.4% of their character test images have
a near-duplicate in training. Re-running their released weights we obtain 95.14% on
their external images but 38.3% on night images; a model trained on our de-duplicated
benchmark reaches 66.7% at night (McNemar p < 0.001) and ties theirs in daylight.
Removing the vehicle stage does not change accuracy and cuts latency by 2.6x.
```

# 16. What to show the faculty (8 slides)

| # | Slide title | What is on it | Source in this report |
|---|---|---|---|
| 1 | The problem | plate parts (city, Metro, letter, numbers); why OCR fails on Bangla; 14-60 classes in older work | Figure 1, section 1 |
| 2 | The paper's method | three YOLOv12 models in a cascade, 104 classes, ordering of labels | Figure 2 (top row) |
| 3 | The paper's results | 95.33 % exact vs 15.33 % OCR; LPD mAP50 0.963, LPNR 0.949; 117 ms on RTX 3050, 2.3 s on CPU | section 5 |
| 4 | We reproduced it | authors' weights on their photos 95.14 vs 95.33; retrained detectors match Tables 7 and 9 | finding 1 |
| 5 | But: its data leaks | 44 % of test photos duplicated in train; 185 of 7,217 crops survive cleaning | Figure 4 |
| 6 | Same photos, two models | tie in daylight (p 0.58), 66.7 vs 38.3 % at night (p < 0.001) | Figure 7 + table |
| 7 | Faster and cleaner | no vehicle stage: 2.6× faster, same accuracy; v1.1 data fix: +4 / +8 photos | Figures 8 and 6 |
| 8 | How it shapes our system | our pipeline (Figure 2 bottom row), format check, next steps | Figure 2, section 20 |

> **Keep it honest:** say "on the same photos" for the comparison with the authors' weights and "on our test split" for the rest. If you show mAP50-95, show Figure 9 with it. Never show the 7.1 FPS row.

# 17. What to explain: a 3-minute script

"This paper is the strongest recent work on Bangladeshi plates, a 2026 PLOS One article from Khulna University. They chain three YOLOv12 models: one finds the vehicle, one the plate, and one finds every character and even whole district names like 'Dhaka' as separate objects, so they need no OCR engine. They report 95 percent of plates read exactly on 150 external photos, against 15 percent for OCR.

We ran their released weights on their own photos with our code and got 95.1 percent, so their result is real and our code is correct. We also found that 44 percent of their plate test photos have a near-copy in their training set, so their test scores are optimistic.

Then we trained the same design on our merged, de-duplicated dataset and compared both models on the same 204 photos. In daylight they tie: the difference is three photos and not significant. At night their model reads 38 percent of plates and ours 67 percent, which is highly significant. We also removed their first model, the vehicle detector: accuracy did not drop and the pipeline became 2.6 times faster.

Our second dataset version fixed 91 images, 63 of them training images, that were whole car photos instead of plate crops. Only that fix raised accuracy on the unchanged test photos by two to four points. And a simple plate-format check catches two thirds of the reading errors without a single false alarm.

So we keep their best idea, the character-level YOLO with word classes, and change the rest: no vehicle stage, a two-row decoder, a format check, tracking for video, and a fair benchmark. That is the plan in our proposal."

## 18. Questions the faculty may ask

**Q1. Why not just use their model?**
It ties ours only in daylight; at night it reads 38 % (ours 67 %). It also uses a slow vehicle stage, a class list with duplicate names, and was tested on a leaky split. We keep its idea, not its weights.

**Q2. Your daylight number is lower than theirs (93.1 vs 95.1). Is your model worse?**
No. On those 144 photos the two models disagree on 13 photos (5 vs 8); McNemar p = 0.58, so this is chance. Without the vehicle stage both read exactly 135. Their photos also come from the same sources as their training data.

**Q3. Why did your numbers change between the two runs?**
Dataset v1.1 fixed 91 BLPD images that were whole photos instead of plate crops. The detector and the end-to-end photos did not change, and the authors' result is identical in both runs, so the +2 to +4 points come from the fix.

**Q4. Why is your detector's mAP50-95 only 46?**
One source, ridoykj (24 % of the test set), draws its boxes tighter than everyone else. Our detector finds those plates (mAP50 99.5) but draws the box around the plate rim. Without ridoykj the score is 64.3, close to their 68.5.

**Q5. Is removing the vehicle stage safe?**
Yes. 7 photos gained, 4 lost, p = 0.55, and it is 2.6× faster. It also rescues dark photos where COCO misses the vehicle and vehicles COCO does not know (CNG, rickshaw).

**Q6. What is McNemar's test and why use it?**
It compares two models on the same photos by counting only the photos where exactly one model is right. It tells whether a gap like 95.1 vs 93.1 could be luck. p below 0.05 means a real difference.

**Q7. How do you know your evaluation code is correct?**
With the authors' weights it reproduces their Table 10 (95.14 vs 95.33 % exact, 98.93 vs 98.82 % characters), and two runs on different days give the same results.

**Q8. Algorithm 1 says "sort left to right". Did you do that?**
Done literally on two-line plates it reads 1 of 651 plates. The paper's own text orders labels by type (city, Metro, letter, number); our decoder does that (87.7 %).

**Q9. Is it real time?**
On a T4 GPU: yes, about 28 photos per second without the vehicle stage, and the detector alone runs at 131.6 FPS with TensorRT. On a CPU: not yet; the detector alone is 6 FPS on Kaggle's CPU. Next we try a nano model and OpenVINO.

**Q10. Is their 15 % OCR baseline fair?**
We cannot tell: the paper does not name the OCR engine. Paper #6's kit runs EasyOCR on our data, so we will report our own OCR baseline.

## 19. Next steps (in order)

1. **Fill the notes and team table:** extract `paper02_outputs.zip` from `After train/results.zip`, then run `python add_results.py paper02_outputs.zip --member "Rokib Hasan Oli"` in `docs/results`. It labels the row v1.1.
2. **Upload** (section 20).
3. **Run** `paper02_original_check.ipynb` on Kaggle (about 7.5 h) to make the reproduction of Tables 6-10 official.
4. **Night:** the detector misses 9 of the 60 NightLPBD plates at confidence 0.25; every "no reading" at night is such a miss. Try a lower threshold at night, night augmentation and more night data.
5. **Format fallback in the cascade:** flag invalid readings (52 of 80 errors) and re-read them with a second reader, as in paper #24.
6. **Speed on a laptop:** YOLO12n/YOLOv8n plate detector, 416-512 px input, ONNX/OpenVINO; measure the full pipeline (≥ 10 FPS target).
7. **Box style:** report mAP50 plus per-source mAP50-95, or re-draw ridoykj boxes in a dataset v1.2 (team decision).
8. **Detector and reader choice (O2):** YOLOv8n / YOLO11n / YOLO12n with equal settings and 3 seeds on our split.
9. **Kit fix for all teammates:** report TensorRT/FP32 FPS (and a median) instead of the PyTorch FP16 mean.

## 20. GitHub and Drive

`After train/results.zip` (448 MB) is the whole Kaggle output; do not upload it. Inside it are the two files the kit made for you. Layout as for paper #23 and Md. Yousuf's `weights/paper03/Md.Yousuf/`:

| Where | What |
|---|---|
| `CSE498R / weights / paper02 / MD.Rokib.Hasan.Oli /` | README.md (main v1.1 results + limits), best_lpd_v12s.pt (18 MB), best_lpnr_v12n.pt, paper02_results.zip (2.3 MB), results_vs_paper.md, this PDF (optional) |
| Team Drive `498R / weights / paper02 /` | paper02_outputs.zip (139.1 MB, with weights, ONNX and TensorRT files) |

Web upload: open the repo's `weights` folder → Add file → Upload files → drag the `paper02` folder → commit message "Add paper #2 results on dataset v1.1 (Rokib)". Files must be under 25 MB each for the web upload.

## Appendix: where every number comes from

| Number | Source |
|---|---|
| paper numbers | PLOS One e0355858, pages given in the text |
| v1.1 training, cascade, condition, speed results | `After train/results.zip` → `runs/*/results.json`, `results.csv`, `logs/*.log`; `paper-2.log` |
| per-photo comparisons, McNemar tests, error types, format check | `report_src/analysis.py` → `analysis.json` (from `e2e_preds_*.csv`, `ocr_preds_test.csv`) |
| per-source mAP, box ratios, Figure 10 | `report_src/persource_det.py`, `ridoykj_figure.py` (local RTX 5060 Ti, the run's best.pt, dataset v1.1) |
| v1.0 numbers | Kaggle log of 19 Oct (paper02-r.log), quoted in `Paper02_Study_Report_v1.0_run.pdf` |
| retraining on BLPD's own split | local preview of 13 Oct 2026 in the paper #2 notes |
| leakage and cleaning | `dataset/docs/leakage_audit.md`, `dedupe_report.md`, `DATASET_v1_1_CHANGES.md` |

```
@article{sarker2026cascaded,
  title   = {Cascaded {YOLO}-based approach for {Bangladeshi} license plate
             identification and number recognition},
  author  = {Sarker, Proshenjit and Alam, S. M. Shamsul},
  journal = {PLOS One},
  volume  = {21}, number = {8}, pages = {e0355858}, year = {2026},
  doi     = {10.1371/journal.pone.0355858}
}
```